HW3 reflection: context engineering for RunX

Yue Li, CS7180. Evidence: comparison.md, changelog.md, `AGENTS.md`.

How the rules file bridges the PRD and mockups to code

The PRD says what to build and the prototype shows how it looks, but an AI tool in the IDE sees neither unless something points to them. AGENTS.md is that pointer and the translation layer. It names the documents to read first (PRD stories, DESIGN.md, the tokens), turns the design into rules the code can obey (use `var(--...)` tokens, 44 px targets, a table of components with expected behavior), and turns the architecture into a folder split (thin pages, logic in `src/lib/`, pure scheduler).

The before/after test on issue #6 shows what that is worth. Given one sentence ("a running app where the user picks a run length"), Copilot built a sensible but generic dark fitness screen: no story, no characters, Arial, 25 hardcoded hex colors, a 38 px profile button and 111 lines inside one page file. With AGENTS.md and the issue it read the issue, DESIGN.md and the tokens first, and produced the RunX home screen: both characters, the glass episode card, length chips with 30 preselected, the finish time, 4 hex colors against 76 token uses, a 5-line page and a component in `src/components/`. It met all three required acceptance criteria against one for the plain prompt. The rules mattered most for design intent and architecture, and least for test quality: tests were similar in both runs, and the rules run even skipped clicking Start and left the E2E spec stale.

An accidental result was equally useful. A first no-rules attempt in the same VS Code window still began every reply with the 🏃 marker, searched for PRD and design files that were not on the branch, and copied the earlier component's copy. The tool had carried rules across chats through session history and attachments. So the marker is a dependable way to check that a tool loaded the file, and a fair "without rules" condition needs a fresh clone, a new window and an empty chat, not just a deleted file.

How the Scrum setup on GitHub helps AI-assisted development

The board, milestones and issues became the unit of context. Issue #6 carried its own acceptance criteria, so the with-rules prompt could be one line and the agent fetched the rest with `gh issue view`. The branch convention (`feature/6-...`) and the PR template gave the agent a finish line, and it created the right branch name unprompted. Sprint 1 issues in "Sprint Todo" made "which feature do we test?" a five-minute choice (I moved from the scheduler, issue #5, to the home screen, #6, because the scheduler depends on an unbuilt manifest and shows no design understanding). CI also paid for itself: it turned red twice on formatting problems in config and template files that would have shipped unnoticed otherwise. What the setup cannot do yet is enforce the process on an AI: neither run wrote a failing test first or committed its work.

What worked, what did not, and what changed from v1 to v2

Worked: explicit read-first lists, concrete numbers (44 px, 30 preselected, file locations), the component table, and the 🏃 marker as a check. Did not work: a scope rule written as prose ("DESIGN.md also describes things... do not build them") was read and still ignored: the agent drew a non-functional world-toggle button. Ambiguous wording also failed: "Files kebab-case; components PascalCase" on one line produced `home-screen.tsx`. Rules that need a process step (TDD order, E2E sync, one test file per module) were skipped.

v2 (129 lines, nine changes, listed with their evidence in the changelog) therefore states behaviors instead of intentions ("don't add controls that do nothing"), adds an explicit naming example, moves validation rules into `src/lib/`, bans color and size literals outside the tokens, requires clicking every primary action in tests, asks the agent to commit each red and green step, and corrects the stack table after the scaffold installed TypeScript 5 and ESLint 9 instead of the newest majors I had listed. I have not re-run the feature with v2, so the improvement from v1 to v2 is a hypothesis; the comparison also rests on one run per condition.

What I would add or change for Sprint 2

1. Re-run the same issue with v2 and measure the rule breaks again (9 and 7 of 14 today).
2. Add scoped rules for the work that is coming: an API and database file with row-level security and error-code rules, and a short audio-content file for the script style guide.
3. Add a pre-commit hook and a CI step that check what prose cannot: no hex colors outside `tokens.css`, a test file for every new module, a Prettier-clean tree.
4. Keep the issue as the single source of acceptance criteria, and add a "files to read" line to each issue so the agent starts from the right context.
5. Run every AI experiment from a fresh clone so context leaks cannot hide the effect of a change.